IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Received 13 January 2026, accepted 26 January 2026, date of publication 12 February 2026, date of current version 18 February 2026.

Digital Object Identifier 10.1109/ACCESS.2026.3661085

RESEARCH ARTICLE

A View-Oriented Skeleton Generation Method for Improving Multi-Table Text-to-SQL Translation Accuracy

HONGYONG ZHOU[1], TIAN TIAN[2], ZIHE DUAN[3], ZESAN LIU[4], DADI WANG[2], AND XIAOWU ZHANG[2]

[1]Marketing Service Center, State Grid Jiangsu Electric Power Company Ltd., Nanjing, Jiangsu 210000, China
[2]Beijing Branch of China Power Industry Group, State Grid Information and Telecommunication Group Company Ltd., Beijing 100052, China
[3]Marketing Service Center, State Grid Hebei Electric Power Company Ltd., Shijiazhuang, Hebei 231000, China
[4]Information Industry Research Institute, State Grid Information and Telecommunication Group Company Ltd., Beijing 100052, China

Corresponding author: Tian Tian (ncepu_ashur0925@sina.com)

This work was supported by the Science and Technology Project of State Grid Corporation of China under Grant 52100125002K-143-ZN.

ABSTRACT Recently, research on natural language to structured query (Text-to-SQL) has advanced rapidly, aiming to enable models to automatically translate natural language queries into executable SQL statements. When handling certain categories of multi-table queries, specifically those that can be abstracted as single-table or natural-join views, models must not only generate SQL syntax but also correctly infer join relationships. To substantially reduce generation difficulty and improve translation accuracy for these query types, this paper proposes a view-level semantic abstraction-based SQL generation method (View-SQL). It first generates an SQL skeleton on a pre-built view, after which a rule-based module completes the table names and the FROM clause to produce the final executable SQL statement. To achieve this, View-SQL introduces two collaborative processing paths. The first is the View Path (Text2SQLSkeleton), where a View Classifier determines which type of view the natural language query corresponds to, and a T5-based decoder generates the SQL skeleton for that view. The second is the Non-View Path (Text2SQL), which is invoked when a query cannot be mapped to any existing view. In this path, the model adopts BERT and multi-head attention mechanisms to select relevant tables and columns before generating the SQL statement. Through this hierarchical framework, View-SQL effectively reduces the complexity of multi-table inference. Experimental results demonstrate that on our validation split, since the Spider test set is not publicly available, View-SQL achieves accuracy improvements of 1.55% and 2.13% on the Spider-syn and Spider EM datasets, respectively.

INDEX TERMS Text-to-SQL, view-oriented SQL generation, multi-table query, view classifier, SQL, multi-head attention.

I. INTRODUCTION

With the continuous advancement of internet technology, data storage demands continue to grow [1], [2]. Text-to-SQL task has gradually emerged as a critical research direction for intelligent question-answering systems and automated data analysis. This task aims to automatically convert natural language queries into executable SQL statements, enabling seamless transition from semantic understanding to data retrieval in human-computer interaction. Finegan-Dollak et al. [3] systematically analyzed the inadequacies of evaluation methods in Text-to-SQL tasks, proposing improvement strategies and standardized datasets to enhance systems' generalization capabilities in real-world scenarios. They introduced a novel data partitioning approach, the query-based split, which more accurately assesses models' generalization performance on unseen SQL patterns. For cross-domain Text-to-SQL tasks, Bogin et al. [4] proposed utilizing graph neural networks (GNNs) to represent database schema structures, thereby enhancing the model's

The associate editor coordinating the review of this manuscript and approving it for publication was Mu-Yen Chen.

understanding of complex relationships and cross-table joins. The database schema is constructed as a directed graph, where tables and columns serve as nodes, and primary/foreign keys and containment relationships form edges. Subsequently, a Gated Graph Neural Network (GGNN) is employed to obtain a global structural representation for each node. They further introduced a question-conditioned relevance score to dynamically adjust node weights based on specific query contexts. Building upon this foundation, Bogin et al. [5] further proposed a global reasoning mechanism that enables the model to more accurately select database constants in zero-shot scenarios. Their dual-path architecture, comprising Gating GCN and Re-ranking GCN, achieved an accuracy improvement from 39.4% to 47.4% on the Spider dataset.

SQLizer [6] is an automated SQL synthesis system that combines natural language processing with program synthesis methods to automatically generate SQL queries from English questions. First, it converts natural language into a Query Sketch, defining only the SQL structure through placeholders such as SELECT and WHERE. Then, it automatically fills in the sketch's blanks using database metadata, ensuring type consistency. Finally, it modifies the sketch structure through error detection and correction rules. A Natural Language to SQL Generation Algorithm Based on Syntactic Dependencies and Database Metadata [7] automatically generates SQL clauses such as SELECT, FROM, and WHERE by parsing dependencies in natural language. It employs Stanford Dependencies to analyze syntactic structures, extracting projection-oriented and selection-oriented constituents.

To enable non-technical users to retrieve database information using natural language, NaLIR [8] is designed to automatically convert natural language queries into executable SQL queries. It employs a three-stage approach. During the parsing stage, the Stanford Dependency Parser converts sentences into dependency syntax trees. The interactive semantic analysis stage maps syntax trees to SQL semantic structures (Query Trees), allowing users to interactively resolve ambiguities. Finally, the validated Query Tree is translated into SQL statements. The Execution-Guided Decoding (EG) framework [9] executes portions of SQL statements in real-time during generation to detect and eliminate semantic or syntactic errors, thereby enhancing model robustness and execution accuracy. It dynamically executes partial results while generating SQL, pruning early upon error detection. Furthermore, it can be embedded into any autoregressive generative model, such as Seq2Seq [10].

Existing Text-to-SQL models exhibit high accuracy when handling single-table queries but experience significant accuracy degradation in multi-table scenarios. To address the limitations of existing research, this paper proposes View-SQL, a view-level semantic abstraction-based SQL generation method. By adopting a two-stage mechanism that first generates an SQL skeleton and then completes the table structure, the method significantly reduces the complexity of multi-table query generation. The main contributions of this paper are as follows.

(1) The concept of database views is introduced into Text-to-SQL tasks, whereby multi-table queries are transformed into single-view queries. By generating SQL skeletons on predefined views, the need for explicit inference of inter-table relationships by the model is reduced.

(2) A dual-path collaborative generation architecture was designed. The View Path (Text2SQLSkeleton) specializes in queries for mappable views, generating SQL skeletons through a view classifier and T5 generator. The Non-View Path (Text2SQL) handles queries that cannot be mapped to predefined views, ensuring compatibility and system-level universality by remaining applicable to the full range of Text-to-SQL queries beyond the view-covered subset.

(3) A rule template module is introduced to automatically complete table names and FROM clauses after skeleton generation, thereby ensuring SQL executability and syntactic integrity while compensating for the structural constraints of the generation model.

II. RELATED WORK

With the maturity of machine learning methods [11], [12], neural networks have shown powerful learning performance and started to be applied to machine translation, giving birth to Neural Machine Translation. Using a bilingual parallel SQL corpus to train the neural network model, the fundamental concept is to take a sentence in one language (the source utterance) as input and use it to produce a corresponding sentence in another language (the target utterance) as output. Encoding the input statement and decoding the encoded output are the two essential phases in neural machine translation. There are two primary SQL translation modes: SQL-to-Text and Text-to-SQL.

A. MACHINE TRANSLATION

MT is an NLP [13], a widely used task that has drawn a lot of interest in recent decades. Automatic translation from a single natural language to a different one is the aim of machine translation. MT techniques can be divided into two main groups: corpus-based and rule-based. Rule-based approaches create the target translation by applying the grammar and linguistic conventions of a particular language pair [14]. Nevertheless, corpus-based techniques, such as instance-based machine translation, NMT [2], and SMT [3], do away with the need for language specialists and the creation of linguistic rules.

B. TEXT-TO-SQL

The objective of the semantic parsing subtask known as ''text-to-SQL query generation'' is to convert natural language text into a comparable formal representation, like a SQL query, logical form, or code generation. A popular method according to published work on text-to-SQL generation is to utilize a sketch based on a SQL structure with numerous slots and define the problem as a slot-filling assignment

by merging it in some way, while employing copying and pointing methods for a wide range of queries. This approach helps to generate accurate SQL queries for natural language problems. The Seq2SQ [15] method is a framework based on augmented pointer networks that take SQL questions, table schemas, and keywords as inputs and mainly utilize the unique structure of SQL to prune the output space of the target query. Instead of employing a sequence-to-sequence method, the SQLNet [16] solution uses a sketch-based method to circumvent the ''order problem'' in conditional sections. The suggested TYPESQL [17] technique, which further refines the SQLNet method, uses type data to capture unusual items and integers in natural language tasks and considers Text-to-SQL issues as slot-filling tasks. Coarse2Fine [18] is a two-stage semantic parsing technique that first creates an outline of a specific issue and then uses the sketch and the input natural language problems to fill in the specifics. The fact that these techniques heavily rely on the framework of dictionaries and SQL is one of their drawbacks.

Text-to-SQL is a task aimed at automatically generating SQL queries from natural language text. This has been a long-standing challenge, as it is important to improve the accessibility of databases so that SQL expertise is no longer required. By automatically generating queries, Text-to-SQL supports the development of session agents with advanced data analysis capabilities. As sequence-to-sequence applications, the language model can be extended to Text-to-SQL tasks. By fine-tuning the SQL-specific design and domain knowledge, the medium-sized models have worked well. They were state-of-the-art for a long time until they were recently surpassed by a contextual learning approach. Among them, PICARD uses incremental parsing to constrain autoregressive decoding; RASAT [19] learns database schema relation-aware self-attention as well as constrained autoregressive decoders; database schema linkage and skeleton parsing are separated by RESDSQL [20] using ranked augmented coding and skeleton-aware decoding frameworks.

C. SQL-TO-TEXT

The objective of the SQL-to-text task is to generate computer-generated descriptions of how a specific Structured Query Language (SQL) query should be interpreted that are legible by humans. This task is important for natural language interfaces to databases because it helps non-expert users understand complex SQL queries that are used to use various text embedding techniques. Early attempts at the SQL-to-text task used rule-based and template-based approaches. Although these approaches require a lot of manual effort to design rules or templates, they tend to generate stiff and mechanical text that lacks natural language features. To address this problem, a sequence-to-sequence (Seq 2Seq) [21] network is used to jointly model SQL queries and natural language. However, since SQL is designed to express graph-structured query intent, sequence encoders may need to be carefully designed to fully capture global structural information. Intuitively, various graph coding techniques based on deep neural networks, whose goal is to learn a node-level or graph-level representation of a given graph, are more suitable for problem solving.

For the SQL-to-text task, a model can be said to have combinatorial generalization capabilities if it learns different ways of combining SQL clauses during training, still understands their meanings, and provides correct translations when confronted with these combinations. In practice, users of the model usually need it to be able to handle complex SQL queries and provide correct translations, while the coverage of complex SQL in the training data may be low. This requires the SQL-to-text model to be able to learn the basics needed for the current application scenario from simple SQL and to be able to understand the query correctly by restructuring the basics when facing complex SQL, which is a reflection of the ability of combinatorial generalization.

III. METHOD

A. OVERVIEW OF THE VIEW-SQL MODEL

In the Spider [22] benchmark dataset for multi-database, multi-table Text2SQL research, databases contain multiple tables regardless of whether the labeled SQL statements represent single-table or multi-table queries. Current studies typically employ neural network methods and sequence modeling approaches to represent database schemas. Existing Text2SQL studies show significantly higher accuracy for single-table SQL translation compared to multi-table queries. This is because when the database schema corresponding to a query contains only one table, the model does not need to learn table-column inclusion relationships or where table names appear within SQL statements.

Therefore, to enhance the accuracy of SQL translation for the model, this paper proposes a view-based Text2SQL method named View-SQL. This approach transforms the model from generating SQL statements on base tables to generating SQL skeletons on views that omit table names and identical clauses. Given that fact-defined views already cover all query scenarios, this paper first establishes views for single-table queries and two-table join queries, the most common scenarios in practical applications. The SQL conversion instance is shown in Figure 1.

The model architecture of the View-SQL method is shown in Figure 2, consisting of the Text2SQLSkeleton model and the Text2SQL model. Both the Text2SQLSkeleton model and the Text2SQL model comprise three components: the input layer, the classification layer, and the Text2SQL layer. The Text2SQLSkeleton model first classifies queries to determine whether they belong to pre-created views. For view queries, the Text2SQLSkeleton model first generates an SQL skeleton on the view, then adds details such as table names and form clauses to progressively construct the complete SQL statement. Non-view queries are handled directly by the Text2SQL model to generate SQL statements.

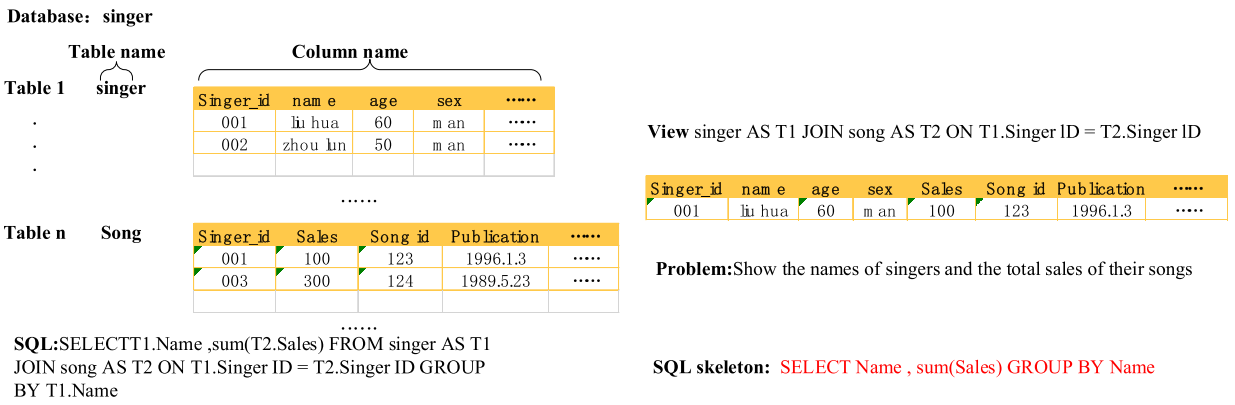

FIGURE 1. An SQL conversion instance.

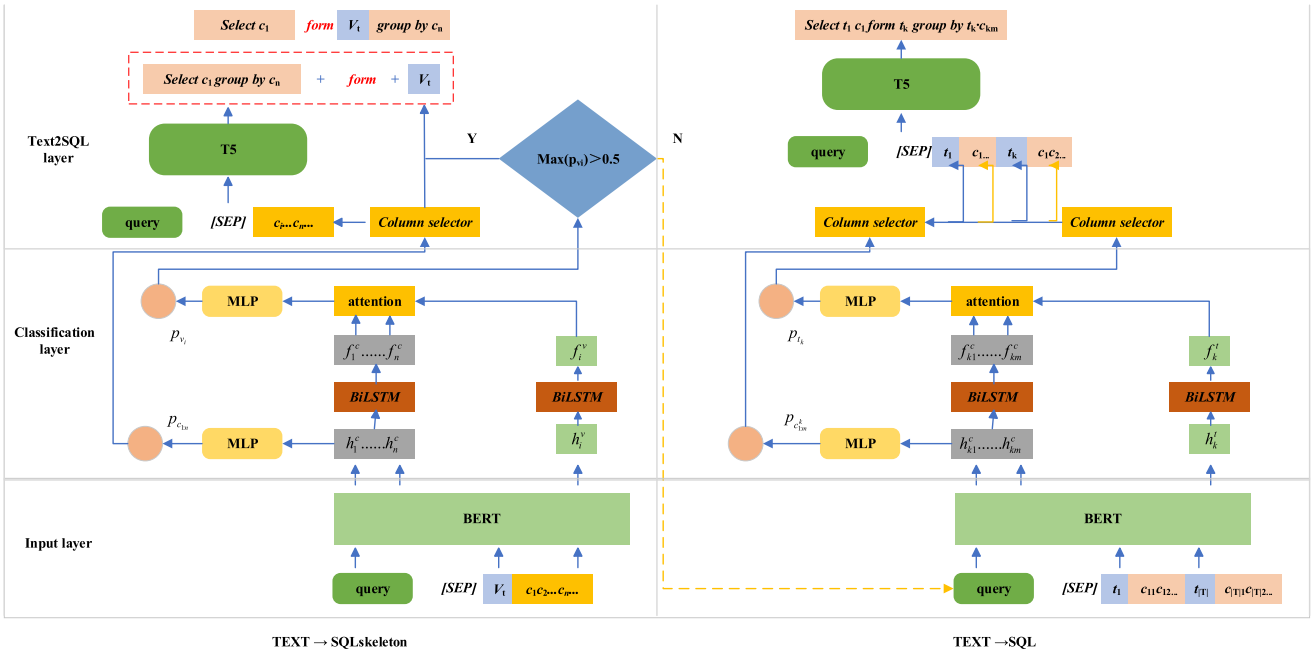

FIGURE 2. Architecture of the view-SQL mode.

B. PROBLEM DEFINITION AND VIEW CREATION

View-SQL takes a given natural language query *query* and a database schema S as input. Here, $query = \{q_1, \ldots, q_{|Q|}\}$ consists of |Q| words, while S primarily contains table names $T = \{t_1, \cdots, t_k \cdots, t_{|T|}\}$, column names $C = \{c_{11}, \ldots, c_{km}, \ldots\}$, and virtual tables $V = v_1, \ldots, v_i, \ldots$ created using primary keys and foreign keys $K = \{(p_1, f_1), \cdots\cdots, (p_x, f_x), \cdots\cdots\}$. Here, |T| represents the number of tables in the current database, t_k denotes the kth table, c_{km} denotes the mth column of the kth table, $C_{vi} = \{c_1, \cdots, c_n \cdots\}$, and c_n denotes the nth column in the ith view.

We make a definition of Join Queries in this paper. We focus on a specific and prevalent class of multi-table queries explicit equi-joins between two tables based on primary foreign key relationships. We may use the term explicit join or join for brevity it should be understood as referring to this pattern such as t_1 *JOIN* t_2 *ON* $t_1.pid = t_2.fid$. This type of join represents a substantial portion of practical multi-table queries in real world databases.

This paper first considers converting the most common query types in practical application scenarios into queries on views, namely single-table queries and two-table join queries, and creates two types of views for them, base tables and virtual tables. For base tables, this section uses the table name as the view name V_i for the current view. Subsequently, the view name v_i, after undergoing word segmentation and stemming, is input into the view classifier, while the columns of this table name serve as the columns of the view. For virtual tables, this paper takes the table names t_1 and t_k corresponding to the primary key and foreign key pair (p_x, f_x), performs stemming on them, and concatenates the results to form the view classifier input v_i. Then, V_i : t_1 *join* t_k *on* $t_1 \cdot p_x = t_k \cdot f_x$ is used as the view name. A FROM

clause is then manually constructed and inserted into the SQL skeleton generated by the Text2SQLSkeleton model to form the complete SQL statement. Subsequently, the column indices of tables t_1 and t_k are input. If a column exists in both tables with the same name, only one is retained. If tables t_1 and t_k contain additional columns, the corresponding base table name is prefixed to each column name.

The views in our current implementation are intentionally constructed to cover single-table queries and two-table explicit joins based on primary-foreign key relationships. This design choice is motivated by the observation that these two patterns represent a significant and well-defined subset of practical multi-table queries. Queries involving three or more tables, non-equi joins, outer joins, or nested structures are not mapped to pre-defined views and are consequently processed by the Non-View Path. This explicit scoping allows our View Path to achieve high accuracy within its domain without being over-extended to less common, more complex patterns.

C. TEXT2SQLSKELETON MODEL

The Text2SQLSkeleton model first classifies queries to determine whether they target views. Once a view-based query is identified, the model uses the column names of the view as input to generate an SQL skeleton. Then, elements such as table names and the FROM clause are added to the skeleton to construct the final SQL statement.

The input layer takes natural language queries and the views created in this chapter as input, organizing them into the following sequence, where v_i and c_n represent the view names and column names after tokenization and stemming, respectively. This sequence is fed into the pre-trained BERT model to obtain the embedding representations for all subwords in the sequence.

$$[CLS]\, query\, [SEP]\, v_i c_1 c_2 \cdots c_n \cdots\cdots [SEP]$$

The classification layer of the Text2SQLSkeleton model, also known as the view classifier, takes the embedding representations $\mathrm{h_i^v}$ of all subwords in the view name v_i and the embedding representations h_n^c of all subwords in the column name c_n as input. These embeddings are then fed into bidirectional $\mathrm{LSTM_1}$ and bidirectional $\mathrm{LSTM_2}$, respectively, as shown in Equations 1 and 2.

$$h_i^v = \left[\overrightarrow{LSTM}\left(\vec{h}_{i-1}^{v}, v_{ij}\right); \overleftarrow{LSTM}\left(\overleftarrow{h}_{i+1}^{v}, v_{ij}\right)\right] \tag{1}$$

$$h_n^c = \left[\overrightarrow{LSTM}\left(\vec{h}_{n(i-1)}^{c}, c_{nj}\right); \overleftarrow{LSTM}\left(\overleftarrow{h}_{n(j+1)}^{c}, c_{nj}\right)\right] \tag{2}$$

where $\mathrm{v_{ij}}$ and c_{nj} denote the embedded representations of the jth subword in the view name and column name, respectively. By concatenating the final hidden states from the forward and backward passes and passing them through a pooling layer, the model obtains the view name embedding representation f_i^v and the column name embedding representation $\mathrm{f_n^c}$, as shown in Equations 3 and 4.

$$f_i^v = g\left[\vec{h}_N^v; \overleftarrow{h}_1^v\right] \tag{3}$$

$$f_n^c = g\left[\vec{h}_N^c; \overleftarrow{h}_1^c\right] \tag{4}$$

After obtaining the embedded representations of the view name and column names, this paper uses $\mathrm{f_i^v}$ as the query vector and applies a weighted aggregation over the embedded representations of all column names and the view name within the view through an attention mechanism. This process yields the final embedding representation x_i^v for the view name, as shown in Equations 5 and 6, where f_n in Equation 6 denotes the corresponding embedding representation of the table name or column name.

$$\alpha_{in} = sfotmax\left(exp\left(f_i^v, f_n^c\right) / \sum exp\left(f_i^v, f_n^c\right)\right) \tag{5}$$

$$x_i^v = \sum \alpha_{in} f_n \tag{6}$$

This paper inputs the final embedded representations of the view names and column names, x_i^v and h_n^c respectively, into the classifier to compute p_{vi} and p_{c_n}. Here, p_{vi} denotes the probability that the information contained in the current view can answer the given natural language query, whereas p_{c_n} represents the probability that the nth column is relevant to the current natural language query, as shown in Equations 7 and 8.

$$p_{vi} = MLP\left(x_i^v\right) \tag{7}$$

$$p_{c_n} = MLP\left(h_n^c\right) \tag{8}$$

Since the number of views varies across different databases, this paper formulates the task as a binary classification problem to determine whether each view or column name is relevant to the current query. During model training, the cross-entropy loss function, a commonly used loss function in classification tasks, is employed. However, the number of views or columns relevant to the current query is typically much smaller than the number of irrelevant ones, leading to a highly imbalanced label distribution with only a small proportion of positive examples. Such imbalance can cause significant training bias and degrade model performance. To address this issue, this paper adopts the focal loss, as shown in Equation 9, to improve the model's capability in handling imbalanced data classification.

$$loss = -\alpha_t \left(1 - p_{y_t}\right)^{\gamma} y_t \log\left(p_{y_t}\right) \tag{9}$$

D. TEXT2SQL MODEL

This paper employs the T5 [23] model to generate SQL skeletons and complete SQL statements. In this study, experiments are conducted using the base version of T5 with 12 layers and 200 million parameters.

This paper uses the following sequence as input for the SQL skeleton generation model, where *query* denotes the current natural language query and c_m represents the columns in the view associated with that query. After sorting and filtering, the 10 columns most relevant to the current query are selected.

$$[CLS]\, query\, [SEP]\, c_1 c_2 \cdots c_m \cdots\cdots [SEP]$$

The generated sequence is shown below, where the portion before the '|' represents intermediate results without column names, and the portion after the '|' constitutes the complete SQL skeleton. After obtaining the SQL skeleton, a FROM clause is appended following the SELECT clause to generate the final executable SQL statement.

$$select\_group\ by\_|selectc_1 group\ by\ c_m$$

The input layer organizes the natural language query *query* together with all table names T and column names C in the database into sequences. Here, query denotes the natural language query, t_k represents the kth table in the database, and c_{km} indicates the mth column of the kth table. The separator '|' is used to distinguish different tables. This sequence is then fed into the pre-trained BERT model to obtain the embedded representations of all segmented subwords.

$$[CLS]\, query\, |t_1 : c_{11}, c_{12} \cdots, |\, t_k \cdots c_{km}, \cdots\cdots [SEP]$$

This paper feeds the embeddings of all subwords in both table names and column names into a bidirectional LSTM to obtain the initial embeddings h_k^t and h_{km}^c for the table names and column names, respectively, as shown in Equations 10 and 11.

$$h_k^t = \left[\overrightarrow{LSTM}\left(\vec{h}_{k-1}^t, t_k\right); \overleftarrow{LSTM}\left(\overleftarrow{h}_{k+1}^t, t_k\right)\right] \tag{10}$$

$$h_{km}^c = \left[\overrightarrow{LSTM}\left(\vec{h}_{k(m-1)}^c, c_{km}\right); \overleftarrow{LSTM}\left(\overleftarrow{h}_{m+1}^c, c_{km}\right)\right] \tag{11}$$

Then, the final hidden states from the forward and backward layers are concatenated and mapped back to their original dimensions, as shown in Equations 12 and 13. Through this process, the model more effectively captures and utilizes contextual information from table and column names, thereby providing more accurate representations for subsequent pattern matching and SQL generation.

$$f_k^t = g\left(\left[\vec{h}_N^t; \overleftarrow{h}_1^t\right]\right) \tag{12}$$

$$f_{km}^c = g\left(\left[\vec{h}_N^c; \overleftarrow{h}_1^c\right]\right) \tag{13}$$

User-submitted natural language queries may omit table names. Therefore, this paper treats the embedded representation of the table name, f_k^t, as the query vector. A multi-head attention mechanism is employed to perform weighted aggregation on the final embedded representations f_{km}^c of all columns belonging to the current table together with the table name's embedded representation f_k^t, thereby obtaining the final embedded representation x_k^t of the table name. Subsequently, the final embedded representations of the table and column names are input into classifiers to determine whether they should appear in the SQL statement, as shown in Equations 14 to 16.

$$p_{c_m^k} = MLP\left(f_{km}^c\right) \tag{14}$$

$$x_k^t = Multihead\ Attention\left(f_k^t; f_{km}^c\right) \tag{15}$$

$$p_{t_k} = MLP\left(x_k^t\right) \tag{16}$$

After obtaining the probabilities of the table and column names associated with the current query, this paper selects the top four tables with the highest probabilities and the top five columns with the highest probabilities from each table as input to the Text2SQL layer.

This paper uses the first sequence shown below as input to the SQL generation model. The sequence includes the current natural language query *query*, the base tables t_k relevant to the query, and the column names belonging to each table t_k that are relevant to the query. Table names and column names are sorted in descending order according to the probabilities output by the model classifier. The output is the second sequence shown below, where the portion following the '|' character represents the executable SQL statement.

$$[CLS]\, query\, [SEP]\, t_1 c_1 c_2 \cdots\cdots |t_k\ \cdots\cdots [SEP]$$

$$select\_from\_groupby\_|\ select\ t_1 :$$

$$c_1\ from\ t_k\ group\ by\ t_k : c_{km}$$

E. RULE-BASED TABLE NAME COMPLETION MODULE

After generating the SQL skeleton, a rule-based module is employed to complete table names and construct the FROM clause. This module operates as follows.

If the query is classified as a single-table view, the table name is directly retrieved from the view definition. If the query corresponds to a join view, the module extracts the participating tables from the view's join condition. The module then inserts the appropriate table names into the FROM clause and prefixes column names with their corresponding table identifiers when ambiguity arises.

The rules are implemented as a deterministic Python function that maps view identifiers to their underlying table structures, ensuring that the final SQL statement is syntactically correct and executable.

IV. EXPERIMENT

A. DATASET AND EXPERIMENTAL SETUP

This paper employs the Spider dataset [22] as the primary benchmark for validating model performance. In addition, the Spider-syn dataset [24] is used to evaluate the model's linguistic diversity and robustness.

Since the Spider test set is not publicly available, this paper cannot pre-create views for the databases included in the test set. Therefore, model performance is evaluated on the validation set, where the 17 databases and their corresponding data from the original training set are used as the validation set for model parameter tuning. The distribution of different data types in the new training set, validation set, and test set is presented in Table 1.

To validate the effectiveness of the View-SQL method, this paper categorizes data from the Spider-syn and Spider datasets into three types. The first type is single-table queries "single", the second type is two-table join queries involving two tables "natural", and the third type is other multi-table queries "other". The focus of this study is on the "single"

and ''natural'' query types, which can be converted into view-based queries.

TABLE 1. Distribution of different types of data in the dataset.

	Single	Nature	Other	All
Training Set	3624	1500	1337	6461
Validation Set	291	151	97	539
Test Set	575	264	195	1034

We implement View-SQL using PyTorch and HuggingFace Transformers. The model configurations are as follows.

BERT-large for embedding, with a hidden size of 1024. BiLSTM with 2 layers and hidden size 512 for sequence modeling. T5-base for SQL skeleton generation, with 12 layers and 200M parameters. Training uses AdamW with a learning rate of 1e-4 for T5 and 5e-5 for BERT, batch size 16, and cosine learning rate scheduling. We use the same optimizer and learning rate for all parameters in the classifier unless otherwise specified. The rule module is implemented as a dictionary mapping view names to their SQL FROM clause templates.

In recent Text2SQL research, the focus has expanded beyond the fundamental semantic parsing component, namely the generation of SQL statements. Therefore, this paper also conducts experiments using execution accuracy (EA) as a metric for evaluating model performance and compares the results with the state-of-the-art method REDSQL. An SQL statement generated by the model is considered correct if its execution results match that of the gold standard SQL query.

This paper trains the classification layer model and the Text2SQL layer model separately, while employing the same training strategy for both. The learning rate increases linearly to its maximum during the first 10% of training steps and then gradually decays according to a cosine schedule. For the Text2SQL layer, the T5 model is initialized with a learning rate of 1e-4 and a batch size of 16, with all other parameters set to their default values. The classification layer model uses the AdamW optimizer and applies dropout to prevent overfitting. During training, BERT-large is fine-tuned using the focal loss function, where α and γ are parameters of the loss. α is set to 0.75 and γ is set to 2. we do not use a frequency-weighted loss term; thus λ is not applicable. During inference, the routing threshold is $\tau = 0.5$. We select the top 10 view columns in the view path, and in the non-view path we select the top 4 tables and the top 5 columns per table. For decoding, we use beam search with a beam size of 4 and set the maximum output length to 256, while the maximum input length is 512. All experiments use a random seed of 42. We train the classifier for 20 epochs and the generators for 30 epochs, with early stopping enabled. Early stopping is applied based on validation EM.

B. THE IMPACT OF CLASSIFIERS AND QUERY TYPES

The View-SQL model employs a view classifier to determine whether a query can be reduced to a query on a specific view. For view queries, View-SQL generates an SQL skeleton based on the corresponding view, whereas for non-view queries, it generates SQL statements directly on the base tables. View classification errors may occur in two scenarios. In the first scenario, the view classifier fails to classify a view query into any existing view. In this case, View-SQL passes the query to the Text2SQL component, which directly generates SQL statements on the underlying base tables. In the second scenario, the view classifier incorrectly assigns the query to an incorrect view. As a result, the FROM clause added by View-SQL to the SQL skeleton becomes incorrect, causing error propagation that leads to an invalid final SQL statement. In the experimental results presented below, the subscript c (correct) indicates that the view classifier correctly classified the view query to its corresponding view, while the subscript z (zero) denotes that the classifier determined the query was not a view query. Table 2 shows the classification accuracy of different classifiers on the Spider-syn dataset.

TABLE 2. Classification accuracy on spider-syn dataset.

Model	Single_c	Single_z	Nature_c	Nature_z	Other
View classifier	88.17%	10.09%	47.35%	46.21%	91.28%

As shown in the experimental results in Table 2, the view classifier proposed in this paper performs well in identifying queries on single-table views and rarely misclassifies difficult examples into incorrect views. For join view queries, although the classifier incorrectly assigns only a small number of queries to the wrong views, the number of examples correctly classified into the corresponding view is also relatively low. The majority of misclassified examples belong to the ''other'' category, which consists of non-view queries.

The classification accuracy on the Spider dataset is shown in Table 3. Both single-table views and join views demonstrate improved classification accuracy, particularly for join views. This improvement occurs because, when natural language queries explicitly mention table and column names from the database schema, the view classifier can more easily determine whether the query targets a view.

TABLE 3. Classification accuracy on the spider validation set.

Model	Single_c	Single_z	Nature_c	Nature_z	Other
View classifier	91.65%	7.48%	61.74%	29.92%	90.26%

Furthermore, we validated the impact of different query types on model execution accuracy. All queries were categorized into three types: single-table view queries (single), join view queries (nature), and other multi-table queries (other). We then compared the execution accuracy of View-SQL and RESDSQL across these three query types. Tables 4 and 5

present the experimental results of the model on the Spider-syn and Spider datasets, respectively. View-SQL achieved execution accuracy improvements of 2.09% for single-table view queries and 2.27% for join view queries, while non-view queries experienced a 2.05% decrease in accuracy.

TABLE 4. Execution accuracy of different query types on the Spider validation set.

Model	Single	Nature	Other	All
RESDSQL	82.43%	68.56%	47.69%	72.34%
ViSQL	84.52%	70.83%	45.64%	73.69%

TABLE 5. Execution accuracy of different query types on spider dataset.

Model	Single	Nature	Other	All
RESDSQL	85.74%	75.00%	63.07%	78.72%
ViSQL	88.00%	77.27%	60.51%	80.08%

As shown in Table 5, the conclusions drawn from experiments on the Spider dataset are consistent with those obtained from the Spider-syn dataset. These results indicate that the performance gains of View-SQL primarily originate from view queries, whereas the accuracy decline for other queries is attributed to the view classifier misclassifying certain non-view queries as view queries.

However, our view classifier exhibits two notable limitations that affect overall performance. One is that the classifier struggles to correctly identify natural-join view queries, while achieving only 61.74% correct classification on the Spider dataset as shown in Table 3. In contrast, single-table view queries achieve 91.65% correct classification. This gap arises because natural-join queries often lack explicit table-name mentions in the natural language question, making view matching more ambiguous.

The other is that it misclassified of ''other'' queries as views. Approximately 9.74% of ''other'' queries are incorrectly classified as view queries as shown in Table 2. When such misclassification occurs, the system erroneously invokes the view-based skeleton generation path, leading to incorrect FROM-clause construction and a subsequent drop in execution accuracy for these cases as reflected in Table 4.

These errors highlight the trade-off introduced by our two-path design. While view-based abstraction improves accuracy for single-table and natural-join queries, it can harm performance when the classifier fails, particularly for natural-join and complex ''other'' queries.

As shown in the efficiency report in Table 6, the inference overhead of View-SQL is primarily determined by the generative model (T5-base), while the additional costs from the routing and rule modules are relatively minor. Specifically, the view classifier + column selector (BERT-large + BiLSTM head) requires only 26 ms per inference. This indicates that the routing decision used to determine whether to follow the View Path has low latency and does not significantly increase the overall system burden. In contrast, skeleton generation for the View Path takes 187 ms, while full SQL generation for the Non-view Path reaches 262 ms. This indicates that when queries cannot be mapped to predefined views and require more complex generation tasks directly within the base table space, the larger decoding length and search space result in higher generation overhead.

The end-to-end results further highlight the efficiency gap between the dual-path designs. When triggering the View Path, the overall inference time was 207 ms. In contrast, triggering the Non-view Path increased the end-to-end inference time to 288 ms by an increase of approximately 81 ms compared to the View Path. This demonstrates that the View Path's strategy of constructing the skeleton first and then completing the rules not only reduces the complexity of generating accurate results but also offers significant efficiency advantages, particularly for high-frequency scenarios like single-table and two-table natural joins. Meanwhile, the rule completion module consumes only 2 ms and contains zero learnable parameters, validating its lightweight nature. It constructs FROM clauses and completes table names with minimal overhead while effectively reducing the structural details that the generation model must directly learn.

In terms of parameter scale, the system's learnable parameters primarily originate from two major backbones: BERT-large with approximately 340 million parameters and T5-base with approximately 220 million parameters. The end-to-end path contains approximately 560 million parameters, reflecting the system's combination of a strong encoder and strong generator to balance routing discrimination and SQL generation capabilities. The overall average end-to-end inference time is 254 ms, indicating that under typical query distributions, the system's overall latency remains within an acceptable range. Furthermore, the routing mechanism enables prioritizing the more efficient View Path for mappable queries, achieving a better efficiency-effectiveness trade-off while ensuring coverage.

TABLE 6. Efficiency report (parameters and inference time).

Component	Backbone	Params	Inference time (ms)
View classifier + column selector	BERT-large + BiLSTM head	~340M	26
View-path generator (skeleton)	T5-base	~220M	187
Non-view path generator (full SQL)	T5-base	~220M	262
Rule-based completion	Python rules	0	2
End-to-end (View Path triggered)	BERT-large + T5-base	~560M	207
End-to-end (Non-view Path triggered)	BERT-large + T5-base	~560M	288
End-to-end (overall average)	BERT-large + T5-base	~560M	254

C. COMPARATIVE EXPERIMENT

Table 7 presents the EM accuracy comparisons of various Text-to-SQL models on the Spider-syn dataset. Overall,

model performance generally improves with increasing structural complexity. Early structure-aware models like GNN achieved EM rates of only around 38-40%. Models incorporating attention mechanisms and context modeling, such as RAT-SQL, improved to 48.2%. Models like S2SQL, which fuse structural and contextual knowledge, consistently achieved rates between 50-61%. The latest RESDSQL and View-SQL demonstrate a clear lead, achieving 67.21% and 68.76%, respectively. Specifically, View-SQL outperformed RESDSQL by 1.55%. Under this semantic variation on the Spider-syn dataset, ViSQL still surpassed RESDSQL. This sufficiently demonstrates that View-SQL's structural constraints enable the model to capture semantic essence rather than relying on superficial pattern matching.

As shown in Table 8, all models exhibit improved performance on the Spider dataset. This improvement occurs because natural language queries in the Spider dataset explicitly reference table and column names from the database schema, allowing the models to more accurately determine which table and column names should appear in the SQL statements. By reducing errors in column name selection, the proposed View-SQL model surpasses RESDSQL by 2.13%.

TABLE 7. Exact matching accuracy on spider-syn dataset.

Model	EM
GNN + ManualMAS	38.20%
IRNet + ManualMAS	39.30%
RAT-SQL	48.20%
S2SQL + Grappa	51.40%
RoSQL + AutoMAS	58.70%
RoSQL + ManualMAS	59.82%
ETAL + CTA	60.40%
ExSQL	60.83%
RESDSQL	67.21%
ViSQL (ours)	**68.76%**

TABLE 8. Exact matching accuracy on spider dataset.

Model	EM
GNN	48.50%
Global-GNN	52.70%
IRnet	61.90%
RAT-SQL	62.70%
ETA	64.50%
BRIDGE	65.50%
LGESQL	67.60%
ExSQL (ours)	67.99%
S2SQL + RoBERTa	71.40%
RESDSQL	73.40%
ViSQL (ours)	**75.53%**

All comparisons with baseline models in this paper adopt exact match accuracy as the evaluation metric. This choice is made because the model either cannot generate values within SQL statements or exhibits limited capability in doing so. Therefore, in experiments that use execution accuracy (EA) as the evaluation metric, this paper employs only RESDSQL, the best-performing model currently not based on paid large language models, as the baseline. Tables 9 and 10 present the experimental results on the Spider-syn and Spider datasets, respectively. The proposed View-SQL model outperforms RESDSQL by 1.35%, which is comparable to its 1.55% improvement in exact match accuracy. These results demonstrate that although the proposed method employs the large language model T5 to generate SQL skeletons rather than complete SQL statements, this distinction does not significantly affect the accuracy of value prediction.

TABLE 9. Execution accuracy on Spider-syn dataset.

Model	EA
RESDSQL	72.34%
ViSQL (ours)	73.69%

On the Spider dataset, the execution accuracy of the proposed View-SQL method improves by 1.36% compared with RESDSQL, which is smaller than the 2.13% improvement observed in exact match accuracy. This result indicates that the natural language queries in the Spider dataset, which contain direct mentions of database schema elements, help models better predict which table and column names will appear in SQL statements. However, this characteristic does not assist in predicting the values within those SQL statements. In summary, the proposed View-SQL method achieves improved execution accuracy on both the Spider-syn and Spider datasets.

TABLE 10. Execution accuracy on Spider dataset.

Model	EA
RESDSQL	78.72%
ViSQL (ours)	80.08%

D. CASE STUDY

The results presented in above experiments show that, for both the Spider-syn and Spider datasets, single-table views achieve higher classification accuracy, whereas join views exhibit relatively lower classification performance. Therefore, this paper analyzes the errors of the view classifier and finds that it tends to misclassify queries of the two types illustrated in Figure 3.

The SQL statement for Question 1 shown in Figure 3 is an explicit join query. However, the high schooler table alone contains all the column names required to answer this question. As a result, the view classifier may incorrectly classify Question 1 as a single-table query on this table. The SQL statement for Question 2, although not an explicit join query, still requires constructing a join view to connect the two tables specified in the join condition.

To more intuitively evaluate the performance gains achieved by the proposed method for generating SQL

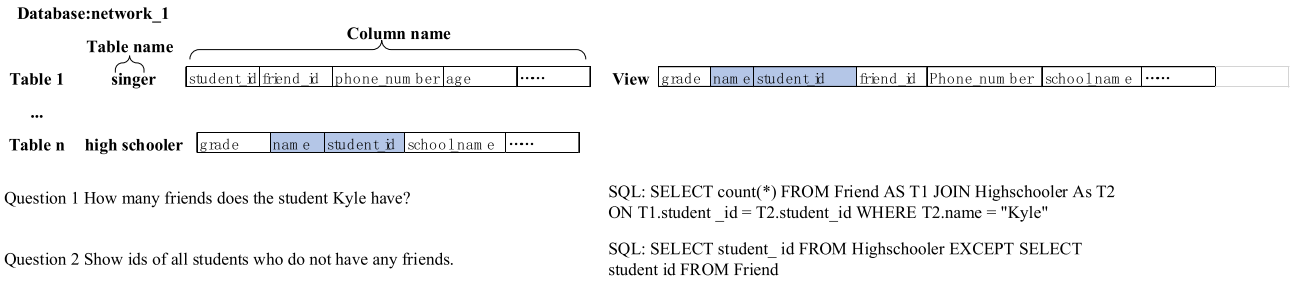

FIGURE 3. View classification case analysis.

skeletons on views, this paper removes the view classifier (VC), which introduces error propagation. Experiments are conducted on the Spider-syn and Spider datasets under the assumption that the view classification results are entirely correct.

As shown in the experimental results in Table 11, these results clearly demonstrate that converting queries into SQL skeletons based on generated views can effectively enhance the Text2SQL accuracy of the model. After converting ''other'' type queries into single-table view queries, the accuracy remains consistent with the high level observed for single-table queries. This occurs because the virtual tables corresponding to join tables are all derived virtual tables. Compared with the base table of a single-table query, the model must select from a larger set of column names, increasing the difficulty of model selection. Furthermore, the number of clauses in non-single-table SQL statements also affects the complexity of model generation.

TABLE 11. Error analysis on spider-syn dataset.

Model	Single	Nature	Other	All
RESDSQL	82.43%	68.56%	47.69%	72.34%
ViSQL w/o VC	86.78%	73.48%	47.69%	76.02%

TABLE 12. Execution accuracy on spider dataset.

Model	Single	Nature	Other	All
RESDSQL	85.74%	75.00%	63.08%	78.72%
ViSQL w/o VC	88.87%	79.55%	63.08%	81.62%

As shown in the experimental results in Table 12, ViSQL achieves SQL translation accuracy improvements of 3.13% and 4.55% for single-type and natural-type data, respectively. These results demonstrate that converting queries into view-based queries yields performance gains regardless of whether the natural language queries explicitly mention table or column names.

V. CONCLUSION

This paper analyzes existing research and finds that single-table queries achieve significantly higher accuracy than multi-table queries in current Text2SQL methods. This is because models do not need to perform logical reasoning over inter-table relationships when generating single-table queries.

To address the low accuracy of multi-table queries in Text-to-SQL tasks, this paper proposes View-SQL, a method for generating SQL skeletons based on view semantic abstraction. Instead of directly generating complete SQL statements on the raw database, View-SQL first constructs SQL skeletons on pre-built views that correspond to single-table or two-table natural-join queries, then completes table names and FROM clauses through rule modules. By doing so, the model only needs to understand column-level semantics during generation without explicitly learning complex multi-table join logic, significantly reducing generation difficulty and enhancing cross-database generalization capabilities. This dual-path collaborative mechanism ensures the system efficiently handles abstractable single-table/two-table join queries. Through experiments on the two publicly available benchmark datasets, using their respective validation splits, the accuracy of View-SQL was validated, showing notable improvements primarily on the single-table and natural-join query categories. It achieved 68.76% EM and 73.69% EA on the Spider-syn dataset and 75.53% EM and 80.08% EA on the Spider dataset. Single-table and join performance improved by 4.35% and 4.92%, respectively. By generating SQL skeletons over predefined views, View-SQL effectively resolves complex semantic modeling challenges in multi-table Text-to-SQL tasks, outperforming state-of-the-art methods on both Spider and Spider-syn datasets.

Nonetheless, the proposed view classifier still faces challenges in identifying join view queries, and the pre-created views cannot yet cover all possible query cases. Therefore, future work will focus on extending the coverage of pre-defined views to include a broader range of join patterns and on improving the view classifier's discrimination capability to better distinguish between view and non-view queries, thereby reducing error propagation and enhancing overall robustness.

ACKNOWLEDGMENT

The authors would like to express that there are no specific acknowledgments for this work.

REFERENCES

[1] Z. Chen, Y. Yi, C. Gan, Z. Tang, and D. Kong, "Scene Chinese recognition with local and global attention," *Pattern Recognit.*, vol. 158, Feb. 2025, Art. no. 111013, doi: 10.1016/j.patcog.2024.111013.

[2] C. Zhao, "Graph adaptive attention network with cross-entropy," *Entropy*, vol. 26, no. 7, p. 576, 2024, doi: 10.3390/e26070576.

[3] C. Finegan-Dollak, J. K. Kummerfeld, L. Zhang, K. Ramanathan, S. Sadasivam, R. Zhang, and D. Radev, "Improving text-to-SQL evaluation methodology," in *Proc. 56th Annu. Meeting Assoc. Comput. Linguistics*, 2018, pp. 351–360, doi: 10.18653/v1/p18-1033.

[4] B. Bogin, M. Gardner, and J. Berant, "Representing schema structure with graph neural networks for text-to-SQL parsing," 2019, *arXiv:1905.06241*.

[5] B. Bogin, M. Gardner, and J. Berant, "Global reasoning over database structures for text-to-SQL parsing," 2019, *arXiv:1908.11214*.

[6] N. Yaghmazadeh, Y. Wang, I. Dillig, and T. Dillig, "SQLizer: Query synthesis from natural language," *Proc. ACM Program. Lang.*, vol. 1, no. OOPSLA, pp. 1–26, Oct. 2017, doi: 10.1145/3133887.

[7] A. Giordani and A. Moschitti, "Generating SQL queries using natural language syntactic dependencies and metadata," in *Proc. 17th Int. Conf. Appl. Natural Lang. Inf. Syst.*, Jun. 2012, pp. 164–170, doi: 10.1007/978-3-642-31178-9_16.

[8] F. Li and H. V. Jagadish, "Understanding natural language queries over relational databases," *ACM SIGMOD Rec.*, vol. 45, no. 1, pp. 6–13, Jun. 2016, doi: 10.1145/2949741.2949744.

[9] C. Wang, K. Tatwawadi, M. Brockschmidt, P.-S. Huang, Y. Mao, O. Polozov, and R. Singh, "Robust text-to-SQL generation with execution-guided decoding," 2018, *arXiv:1807.03100*.

[10] Z. Li, J. Cai, S. He, and H. Zhao, "Seq2seq dependency parsing," in *Proc. 27th Int. Conf. Comput. Linguistics*, Aug. 2018, pp. 3203–3214. [Online]. Available: https://aclanthology.org/C18-1271/

[11] Z. Chen, "(HTBNet)arbitrary shape scene text detection with binarization of hyperbolic tangent and cross-entropy," *Entropy*, vol. 26, no. 7, p. 560, Jun. 2024, doi: 10.3390/e26070560.

[12] Z. Chen, "Arbitrary shape text detection with discrete cosine transform and CLIP for urban scene perception in ITS," *IEEE Trans. Intell. Transp. Syst.*, early access, Feb. 10, 2025, doi: 10.1109/TITS.2025.3534291.

[13] E. Cambria and B. White, "Jumping NLP curves: A review of natural language processing research [review article]," *IEEE Comput. Intell. Mag.*, vol. 9, no. 2, pp. 48–57, May 2014, doi: 10.1109/MCI.2014.2307227.

[14] S. Saini and V. Sahula, "A survey of machine translation techniques and systems for Indian languages," in *Proc. IEEE Int. Conf. Comput. Intell. Commun. Technol.*, India, Feb. 2015, pp. 676–681, doi: 10.1109/CICT.2015.123.

[15] J. Herzig, P. Shaw, M.-W. Chang, K. Guu, P. Pasupat, and Y. Zhang, "Unlocking compositional generalization in pre-trained models using intermediate representations," 2021, *arXiv:2104.07478*.

[16] X. Xu, C. Liu, and D. Song, "SQLNet: Generating structured queries from natural language without reinforcement learning," 2017, *arXiv:1711.04436*.

[17] T. Yu, Z. Li, Z. Zhang, R. Zhang, and D. Radev, "TypeSQL: Knowledge-based type-aware neural text-to-SQL generation," 2018, *arXiv:1804.09769*.

[18] A. E. Eshratifar, D. Eigen, M. Gormish, and M. Pedram, "Coarse2Fine: A two-stage training method for fine-grained visual classification," *Mach Vis. Appl.*, vol. 32, no. 2, pp. 1–9, Mar. 2021, doi: 10.1007/s00138-021-01180-y.

[19] T. Bocklisch, J. Faulkner, N. Pawlowski, and A. Nichol, "Rasa: Open source language understanding and dialogue management," 2017, *arXiv:1712.05181*.

[20] H. Li, J. Zhang, C. Li, and H. Chen, "RESDSQL: Decoupling schema linking and skeleton parsing for text-to-SQL," 2023, *arXiv:2302.05965*.

[21] Y.-J. Tang, Y.-H. Pang, and B. Liu, "IDP-Seq2Seq: Identification of intrinsically disordered regions based on sequence to sequence learning," *Bioinformatics*, vol. 36, no. 21, pp. 5177–5186, Jul. 2020, doi: 10.1093/bioinformatics/btaa667.

[22] T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang, Z. Li, J. Ma, I. Li, Q. Yao, S. Roman, Z. Zhang, and D. Radev, "Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task," 2018, *arXiv:1809.08887*.

[23] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the limits of transfer learning with a unified text-to-text transformer," 2019, *arXiv:1910.10683*.

[24] Y. Gan, X. Chen, Q. Huang, M. Purver, J. R. Woodward, J. Xie, and P. Huang, "Towards robustness of text-to-SQL models against synonym substitution," 2021, *arXiv:2106.01065*.

HONGYONG ZHOU is currently with the Marketing Service Center, State Grid Jiangsu Electric Power Company Ltd., Nanjing, Jiangsu, China, as a Graduate Engineer. His main research interests focus on power system marketing management, smart grid operation, and data-driven service optimization.

TIAN TIAN was born in Zhengzhou, Henan, China, in March 1970. He received the bachelor's and master's degrees in power system and automation from Tsinghua University, in 1993 and 1997, respectively. He holds a senior professional title. He is with Beijing China-Power Information Technology Company Ltd., Beijing. His research areas include product development for power system automation, digitalization, and intelligence.

ZIHE DUAN is currently with the Marketing Service Center, State Grid Hebei Electric Power Company Ltd., Shijiazhuang, Hebei, China, as a Graduate Engineer. Her research interests include power marketing management, smart grid operation, and energy data analytics.

ZESAN LIU received the bachelor's degree in management and information systems from China Three Gorges University, in 2010, and the master's degree in computer application from North China Electric Power University, in 2013. He is currently a Department Leader with the Information and Communication Research Institute, State Grid Information and Telecommunication Group Company Ltd. He mainly engaged in core technology research and development in areas, such as basic common platforms and power grid informatization.

DADI WANG is currently with Beijing Branch of China Power Industry Group, State Grid Information and Telecommunication Group Company Ltd., Beijing, China, as a Graduate Engineer. His research interests focus on information and communication technologies in smart grids, power data management, and intelligent operation systems.

XIAOWU ZHANG was born in Wuhan, Hubei, China, in March 1994. He received the degree in computer science from Jianghan University, in 2016. He holds the intermediate professional title. He is engaged in the design of power system application software and research on enterprise digital technology with the Beijing China-Power Information Technology Company Ltd., Beijing.